

Weight Initialization in Neural Networks

Weight initialization is a critical step in training deep neural networks. The right initialization helps:

- Prevent **vanishing/exploding gradients**
 - Ensure **symmetry breaking**
 - Enable **faster convergence**
-

Working of Neural Networks – Step-by-Step

1. **Initialize Parameters:** Assign initial values to all weights (W) and biases (b).
 2. **Choose an Optimization Algorithm:** e.g., SGD, Adam, RMSprop, etc.
 3. **Repeat Until Convergence:**
 - (a) **Forward Propagation:** Compute layer-wise activations.
 - (b) **Compute Cost Function:** Evaluate prediction loss.
 - (c) **Backpropagation:** Compute gradients of cost w.r.t. weights.
 - (d) **Update Parameters:** Use gradients to update weights using the chosen optimizer.
-

Why Initialization Matters

Improper weight initialization can lead to the following issues:

- **Vanishing Gradients:** Gradients become too small to make useful updates.
 - **Exploding Gradients:** Gradients become too large, leading to instability.
 - **Symmetry Breaking Failure:** If all neurons start with the same values, they learn the same features.
 - **Slow Convergence:** Small or large weights make learning inefficient.
-

What NOT To Do

Method	Problem
Zero Initialization	All neurons start identically → no symmetry breaking. No learning.
Constant Initialization	Still symmetric → same update for all neurons.

Method	Problem
Very Small Weights	Can lead to vanishing gradients (especially with sigmoid/tanh).
Very Large Weights	Can lead to exploding gradients (especially with ReLU) → unstable training.

✓ Practical Initialization Techniques

1. Xavier / Glorot Initialization

- **Best for:** Sigmoid / Tanh
- **Goal:** Maintain equal variance of activations and gradients across layers.
- **Formulas:**

Normal Distribution:

$$W \sim \mathcal{N}\left(0, \frac{1}{n_{\text{in}}}\right)$$

Uniform Distribution:

$$W \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right]$$

2. He Initialization

- **Best for:** ReLU / Leaky ReLU
- **Goal:** Maintain large enough activations to avoid vanishing gradients.
- **Formulas:**

Normal Distribution:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

Uniform Distribution:

$$W \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right]$$

Initialization	Distribution Type	Formula	Use Case
Xavier (Glorot)	Normal	$W \sim \mathcal{N}\left(0, \frac{1}{n_{\text{in}}}\right)$	Sigmoid / Tanh
	Uniform	$W \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right]$	
He	Normal	$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$	ReLU / Leaky ReLU
	Uniform	$W \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right]$	
Zero Init	-	$W = 0$	✗ Never use
Constant Init	-	$W = c \neq 0$	✗ Still symmetric

Python Code: Weight Initialization Examples

```
import numpy as np

# Xavier Initialization (Uniform)
def xavier_init(shape):
    limit = np.sqrt(6 / sum(shape))
    return np.random.uniform(-limit, limit, size=shape)

# He Initialization (Normal)
def he_init(shape):
    std_dev = np.sqrt(2 / shape[0])
    return np.random.normal(0, std_dev, size=shape)

# Example usage:
W1 = xavier_init((256, 128))
W2 = he_init((256, 128))

print("Xavier Init - Mean:", W1.mean(), "Std:", W1.std())
print("He Init      - Mean:", W2.mean(), "Std:", W2.std())
```



Batch Normalization



What is Batch Normalization?

Batch Normalization (BN) is a **technique to normalize the activations of each layer** in a neural network across a mini-batch. It helps:

- Accelerate training.
 - Make optimization more stable.
 - Reduce sensitivity to initialization and learning rates.
-



Key Idea

In neural networks, **each layer's output (activations)** is passed to the next layer as input. Batch normalization normalizes these activations for each mini-batch to have:

- **Mean ≈ 0**
- **Standard Deviation ≈ 1**

This ensures a stable distribution of data across layers, preventing issues like internal covariate shift.



Why Use Batch Normalization?



Internal Covariate Shift

- Refers to changes in the distribution of hidden layer activations during training as weights update.
- BN reduces this shift, keeping the distribution more stable.



Benefits of Batch Normalization

Advantage	Explanation
✓ Faster Training	Allows the use of higher learning rates without risk of divergence.
✓ More Stable	Reduces sensitivity to weight initialization.
✓ Regularization	Acts as a mild regularizer , helping reduce overfitting.
✓ Less Dependence on Init	Reduces reliance on perfect weight initialization methods.

Advantage	Explanation
 Reduces Vanishing Gradients	Especially in deep networks.

How It Works – Step-by-Step

For each neuron activation x in a mini-batch:

1. Compute Mean and Variance:

$$\mu = \frac{1}{m} \sum_{i=1}^m x_i \quad ; \quad \sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$$

2. Normalize:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

3. Scale and Shift (using learnable parameters γ, β):

$$y_i = \gamma \hat{x}_i + \beta$$

- ϵ : small constant for numerical stability
- γ, β : **learned** during training

Parameters in Batch Normalization

For a layer with **n neurons**, Batch Normalization introduces **4 parameters per neuron**:

Learnable Parameters:

- γ (scale)
- β (shift)

These are **trainable** and updated during backpropagation.

Non-learnable Parameters:

- Running mean: μ
- Running variance: σ^2

These are **computed during training** and **used during inference**.

Example: For 3 neurons

- Learnable: $3 \times 2 = 6$
 - Non-learnable: $3 \times 2 = 6$
-

During Training vs Inference

- **Training:** Use batch-wise mean and variance
 - **Inference:** Use **running (moving average)** of mean and variance from training
-

Keras Implementation

```
from keras.models import Sequential
from keras.layers import Dense, BatchNormalization

model = Sequential()
model.add(Dense(3, activation='relu', input_dim=2))
model.add(BatchNormalization()) # After activation
model.add(Dense(2, activation='relu'))
model.add(BatchNormalization())
model.add(Dense(1, activation='sigmoid'))
```

Parameters in Batch Normalization

For a layer with **n neurons**, Batch Normalization introduces **4 parameters per neuron**:

Learnable Parameters:

- γ (scale)
- β (shift)

These are **trainable** and updated during backpropagation.

Non-learnable Parameters:

- Running **mean** μ
- Running **variance** σ^2

These are **calculated and stored** during training, and **used during inference** to normalize activations.

Example:

If a layer has **3 neurons**, then:

- **Learnable** parameters = $3 \times 2 = 6$
 - **Non-learnable** parameters = $3 \times 2 = 6$
-

✓ Summary

Feature	Description
🧠 Faster Training	Allows use of higher learning rates
📧 Stable Gradients	Helps prevent vanishing or exploding gradients
⚙️ Reduced Init Sensitivity	Reduces reliance on careful weight initialization
🔒 Acts as Regularizer	Provides regularization effect (may reduce dropout)
🌐 Improved Generalization	Often improves test performance and reduces overfitting

💡 **Tip:** BatchNorm is typically applied **after the activation** (Dense → Activation → BatchNorm) but can also be applied **before activation** in some architectures.

In []: